

BÁO CÁO BÀI THỰC HÀNH

PHÂN ĐOẠN VÀ PHÂN LỚP ẢNH Y KHOA

Môn: Thị giác máy tính nâng cao

Nhóm: **Group_07**

Nguyễn Thái Thông | Trần Hà Khánh Duy

1. Thông tin nhóm

Nội dung	Thông tin
Nhóm	Group_07
Thành viên 1	Nguyễn Thái Thông
Thành viên 2	Trần Hà Khánh Duy
Bài toán được giao	Segmentation ảnh X-quang y khoa, phân đoạn vùng gãy xương
Dataset được phân công	FracAtlas/segmentation
Mô hình được phân công	U-Net++ với encoder EfficientNet-B3 pretrained ImageNet

2. Phân công công việc trong nhóm

Thành viên	Công việc phụ trách	Tỷ lệ đóng góp	Ghi chú
Nguyễn Thái Thông	Chuẩn bị pipeline đọc dữ liệu, cấu hình mô hình U-Net++, chạy baseline resize, tune threshold/postprocess, kiểm tra metric chính và viết phần phương pháp - kết quả - phân tích.	50%	Phụ trách pipeline, mô hình, huấn luyện baseline và tổng hợp báo cáo.
Trần Hà Khánh Duy	Kiểm tra split dữ liệu, hỗ trợ chạy option no-resize, xuất hình overlay, đối chiếu file kết quả, các nhóm case và hoàn thiện nhận xét.	50%	Phụ trách kiểm tra dữ liệu, hình minh họa, no-resize và rà soát kết quả.

3. Mô tả đề bài và dữ liệu

Bài toán nhóm thực hiện là **phân đoạn nhị phân** vùng gãy xương trên ảnh X-quang. Mỗi ảnh đi kèm một mask. Nếu ảnh thuộc nhóm *fracture*, mask đánh dấu vùng gãy cần tìm; nếu ảnh thuộc nhóm *normal*, mask đúng sẽ là mask rỗng (trạng thái xương bình thường, không có vùng gãy). Dataset được phân công là [FracAtlas/segmentation](#); nhóm dùng đúng các file `train.csv`, `val.csv`, `test.csv` có sẵn và **không tự chia lại dữ liệu**.

Dataset được chuẩn bị dưới dạng file nén `segmentation.zip`. Khi chạy trên Colab, nhóm copy file từ Drive về `/content/segmentation.zip` rồi giải nén vào `/content/segmentation_extracted/segmentation`. Mục đích của bước này là tăng độ ổn định khi chạy thí nghiệm: đọc nhiều ảnh nhỏ trực tiếp từ Drive có thể chậm hoặc bị ngắt, trong khi đọc từ ổ local của Colab thường ổn định hơn.

Cấu trúc dữ liệu sau khi giải nén:

```
segmentation/
|-- train.csv
|-- val.csv
|-- test.csv
|-- images/
|   |-- IMGxxxxxxx.jpg
|   |-- ...
|-- masks/
|   |-- IMGxxxxxxx_mask.png
|   |-- ...
```

Các file CSV chứa đường dẫn ảnh/mask và metadata, gồm các cột chính như `image_id`, `image_path`, `mask_id`, `mask_path`, `label_binary`, `label_3class`, `original_width`, `original_height`, `mask_area`, `split` và `note`. Trước khi train, nhóm cũng đã tiến hành kiểm tra lại đường dẫn ảnh/mask trong CSV; kết quả không có dòng nào bị thiếu file hoặc lỗi đọc.

Tập dữ liệu	Train	Val	Test	Nhân/lớp	Ghi chú
Raw split	2858 ảnh: 502 fracture, 2356 normal	612 ảnh: 107 fracture, 505 normal	613 ảnh: 108 fracture, 505 normal	2 lớp	Split chính thức của dataset.
Train dùng cho base-line	2858 ảnh sau oversampling: 1572 fracture, 1286 normal	612 ảnh	613 ảnh full test	2 lớp	Oversampling chỉ áp dụng trên train, không dùng val/test.
No-resize	192 ảnh train	128 ảnh val	160 ảnh test subset: 23 fracture, 137 normal	2 lớp	Giữ ảnh gốc, nhưng chỉ chạy subset do giới hạn tài nguyên.

Theo quan sát của nhóm thì tập dữ liệu bị lệch khá rõ: trong raw train, ảnh *fracture* chỉ chiếm khoảng **17.6%**. Vùng gãy cũng thường rất nhỏ so với cả ảnh X-quang. Vì vậy nếu chỉ nhìn metric trung bình trên toàn bộ test, kết quả có thể bị ảnh hưởng mạnh bởi nhóm ảnh *normal* có mask rỗng. Trong phần kết quả, nhóm **tách riêng metric trên các ảnh thật sự có mask fracture** để đánh giá rõ hơn khả năng phân đoạn.

4. Phương pháp và mô hình

Mô hình được phân công trong bài thực hành của nhóm là **U-Net++**; nhóm triển khai lại bằng PyTorch để huấn luyện và đánh giá trên dataset được giao. Mô hình gồm *encoder pretrained* để lấy đặc trưng từ ảnh X-quang và *decoder U-Net++* để dựng lại bản đồ phân đoạn. Đầu ra cuối cùng là một kênh *logit*; sau đó nhóm dùng *sigmoid* và *threshold* để chuyển thành mask nhị phân.

Framework và thư viện

Thành phần	Vai trò trong bài làm
Python, PyTorch	Xây dựng mô hình, huấn luyện, đánh giá, tính loss và metric. Môi trường thực nghiệm dùng Colab Notebook - PyTorch 2.11.0+cu128 và CUDA GPU T4.
<code>timm</code>	Tạo encoder EfficientNet-B3 pretrained ImageNet dưới dạng feature extractor.
PIL	Đọc ảnh grayscale, đọc mask, resize/rotate/flip và augmentation cơ bản.
NumPy, pandas	Xử lý array, đọc CSV, tổng hợp kết quả và xuất file <code>results_Group07.csv</code> .
SciPy	Hỗ trợ xử lý connected components trong bước postprocess mask.
Matplotlib	Vẽ hình minh họa gồm ảnh đầu vào, GT mask, prediction và overlay.

Encoder pretrained

Encoder sử dụng `efficientnet_b3` từ thư viện `timm`, bật `pretrained=True` để dùng trọng số ImageNet. Vì ảnh X-quang là grayscale, nhóm chuyển ảnh thành 3 kênh bằng cách lặp lại kênh grayscale, sau đó chuẩn hóa theo mean/std ImageNet. Encoder được cấu hình với `features_only=True` và lấy 5 mức feature map qua `out_indices=(0,1,2,3,4)`. Các mức feature này có số channel lần lượt là:

[24, 32, 48, 136, 384].

Encoder được *freeze* trong epoch đầu để ổn định huấn luyện, sau đó *unfreeze* để *fine-tune* cùng decoder.

Decoder U-Net++

Decoder dùng cấu trúc U-Net++ với các *skip connection* lồng nhau. Cách thiết kế này giúp mô hình vừa giữ được chi tiết ở tầng nông, vừa tận dụng đặc trưng ngữ nghĩa ở tầng sâu. Điều này khá phù hợp với bài toán *fracture* vì vùng gãy thường nhỏ và biên không rõ. Mỗi block decoder gồm *convolution*, GroupNorm, SiLU và Dropout2d. Nhóm cũng bật *deep supervision average*, tức là lấy trung bình một số nhánh đầu ra trung gian thay vì chỉ dùng output cuối, để quá trình học của decoder ổn định hơn.

Hai cấu hình decoder được dùng:

- Baseline `resize_256`: decoder channels [48, 96, 192, 384], trainable params khoảng 16.7M.
- Option `no_resize_original`: decoder channels [16, 32, 64, 96] để giảm bộ nhớ khi input giữ kích thước gốc.

Hàm mất mát

Loss là tổ hợp giữa **Focal BCE with logits** và **Tversky loss**:

$$\mathcal{L} = 0.35 \times \mathcal{L}_{\text{Focal BCE}} + 0.65 \times \mathcal{L}_{\text{Tversky}}.$$

Trong đó Focal BCE dùng $\alpha = 0.85$, $\gamma = 2.0$. Tversky loss dùng $\alpha = 0.25$, $\beta = 0.75$. Focal BCE giúp giảm ảnh hưởng của pixel dễ, còn Tversky loss phù hợp với *segmentation* mất cân bằng vì vùng *fracture* nhỏ hơn rất nhiều so với nền.

Optimizer, scheduler và cấu hình huấn luyện

Thành phần	Cấu hình
Optimizer	AdamW, chia hai param group: encoder LR 3×10^{-5} , decoder LR 3.5×10^{-4} .
Weight decay	10^{-4} .
Scheduler	CosineAnnealingLR, <code>eta_min = DECODER_LR * 0.05</code> .
Mixed precision	Bật AMP để giảm VRAM và tăng tốc trên GPU.
Gradient accumulation	Baseline dùng accumulation 2; no-resize dùng accumulation 4 do batch size 1.
OOM handling	Với no-resize, nhóm cho phép bỏ qua batch bị OOM để quá trình huấn luyện không dừng giữa chừng.

5. Tiền xử lý dữ liệu

Pipeline đọc ảnh bằng PIL ở chế độ grayscale. Mask được chuyển thành nhị phân, trong đó pixel thuộc vùng fracture là 1 và nền là 0. Vì encoder pretrained trên ImageNet nhận ảnh 3 kênh, nhóm lặp lại kênh grayscale thành 3 kênh rồi chuẩn hóa theo mean $[0.485, 0.456, 0.406]$ và std $[0.229, 0.224, 0.225]$ của ImageNet.

Với baseline `resize_256`, ảnh và mask được đưa về kích thước 256×256 . Ảnh dùng *bilinear interpolation*; mask dùng *nearest-neighbor* để tránh sinh ra các giá trị mask trung gian. Với mode `no_resize_original`, nhóm giữ nguyên kích thước ảnh gốc: **không resize vuông, không padding và không giới hạn cạnh dài**. Ở đây, *giới hạn cạnh dài* nghĩa là giới hạn cạnh lớn nhất của ảnh về một ngưỡng cố định, ví dụ ảnh quá lớn thì thu nhỏ theo đúng tỉ lệ để cạnh dài không vượt quá 1024 hoặc 1536 pixel. Nhóm cũng tiến hành thử nghiệm với bước giới hạn nhưng hiện tại quyết định không dùng bước này mà giữ nguyên ảnh gốc, nên ảnh đi vào model đúng theo kích thước gốc; đổi lại batch size phải đặt bằng 1 vì mỗi ảnh có một kích thước khác nhau.

Augmentation chỉ áp dụng cho train và giữ ở mức vừa phải:

- **Horizontal flip** với xác suất 0.5.
- **Vertical flip** nhẹ với xác suất 0.15.
- **Rotate** trong khoảng -7° đến 7° với xác suất 0.25.
- **Autocontrast** với xác suất 0.25.
- **Chỉnh contrast** trong khoảng 0.80-1.30 với xác suất 0.40.
- **Chỉnh brightness** trong khoảng 0.88-1.14 với xác suất 0.35.

Train, chọn threshold và hậu xử lý

Nhóm thực nghiệm với hai mode: `no_resize_original` và `resize_256`. No-resize được dùng để kiểm tra hướng giữ ảnh gốc, còn **resize 256 là baseline chính**. Với mỗi mode, nhóm dùng pipeline dữ liệu và cấu hình mô hình riêng, chọn checkpoint tốt nhất theo *balanced validation score*. Threshold được chọn trên `val.csv`, sau đó mới đánh giá trên `test.csv` hoặc test subset.

Balanced validation score trong giai đoạn monitor được tính theo hướng ưu tiên ảnh fracture nhưng

vẫn kiểm tra ảnh normal:

$$\text{score} = 0.70 \times \text{Dice}_{\text{positive}} + 0.30 \times \text{NormalEmptyRate}.$$

Trong công thức này, $\text{Dice}_{\text{positive}}$ là Dice trung bình chỉ tính trên các ảnh có mask *fracture* thật. Đây là **phần quan trọng nhất** vì mục tiêu chính của bài toán là phát hiện và phân đoạn đúng vùng gãy. NormalEmptyRate là tỷ lệ ảnh *normal* được mô hình dự đoán rỗng đúng, tức là không sinh mask giả trên ảnh không có *fracture*. Nhóm đặt trọng số **0.70** cho Dice positive và **0.30** cho NormalEmptyRate để ưu tiên khả năng bắt fracture, nhưng vẫn không bỏ qua vấn đề *false positive* trên nhóm normal.

Cách tính này phù hợp hơn so với chỉ dùng Dice trung bình toàn bộ validation, vì tập dữ liệu có nhiều ảnh normal mask rỗng. Nếu mô hình dự đoán rỗng cho gần như tất cả ảnh, nó có thể đạt điểm khá trên normal nhưng lại không phát hiện được fracture. Khi đó $\text{Dice}_{\text{positive}} \approx 0$ và $\text{NormalEmptyRate} \approx 1$, score sẽ xấp xỉ 0.30. Đây là lý do trong thí nghiệm no-resize, việc score đứng quanh 0.30000 không được xem là kết quả tốt; nó cho thấy mô hình chủ yếu dự đoán rỗng đúng trên ảnh normal, nhưng chưa học được vùng fracture.

Điểm score này chỉ dùng trong quá trình theo dõi validation để chọn checkpoint và threshold. Khi báo cáo kết quả cuối, nhóm vẫn trình bày đầy đủ Dice, IoU, Precision và Recall trên test, đồng thời tách riêng kết quả trên nhóm ảnh có mask fracture thật để tránh hiểu sai do dữ liệu lệch normal/fracture.

Sau khi model dự đoán xác suất pixel bằng sigmoid và threshold thành mask nhị phân, nhóm có thêm bước hậu xử lý. Bước này lọc connected components quá nhỏ và kiểm soát tổng diện tích mask dự đoán. Mục tiêu là loại bỏ các vùng nhiễu rời rạc, đặc biệt trong những ảnh normal dễ sinh mask giả ở biên xương hoặc vùng tương phản mạnh.

Riêng baseline resize dùng thêm *image-level gate* để giảm *false positive* trên ảnh *normal*. Gate này không thay đổi kiến trúc mô hình, mà là một bước quyết định ở mức ảnh sau khi có bản đồ xác suất. Ý tưởng là: nếu xác suất lớn nhất trong ảnh còn thấp, hoặc tổng vùng xác suất cao quá nhỏ, thì dự đoán đó chưa đủ tin cậy để giữ lại mask; khi đó mask được đặt về rỗng. Bộ tham số tốt nhất trên validation resize là `seg_threshold=0.08`, `gate_max_prob_threshold=0.55`, `gate_min_soft_area_ratio=0.0005`. Nói đơn giản, một ảnh chỉ được giữ mask khi mô hình vừa có điểm tin cậy đủ cao, vừa có một vùng nghi ngờ đủ rõ về diện tích mềm.

Gate có ưu điểm là giảm một phần false positive trên ảnh normal, nhưng cũng có trade-off. Nếu đặt gate quá chặt, các fracture nhỏ hoặc mờ có thể bị xóa nhầm, làm tăng false negative. Vì vậy nhóm chỉ dùng gate cho baseline resize sau khi sweep trên validation. Với no-resize, nhóm tắt gate để tránh việc vùng fracture nhỏ trên ảnh gốc bị loại bỏ sớm; trong mode này, nhóm muốn quan sát trực tiếp khả năng mô hình sinh tín hiệu fracture trước khi áp dụng thêm một tầng lọc ở mức ảnh.

Mode	Epoch	Batch	Threshold	Ghi chú thực nghiệm
<code>resize_256</code>	40	12, accum 2	0.08 + gate	Checkpoint tốt nhất ở epoch 36, điểm theo dõi 0.459421; đánh giá trên toàn bộ 613 ảnh test.
<code>no_resize_original</code>	6	1, accum 4	0.03, không gate	Checkpoint tốt nhất ở epoch 1; đánh giá trên subset 160 ảnh; một số batch OOM được bỏ qua ở các epoch sau.

6. Kết quả baseline resize

Baseline resize là phần bắt buộc của bài làm. Nhóm huấn luyện mode này trong **40 epoch** và đánh giá trên toàn bộ **613 ảnh** trong `test.csv`. Các metric chính gồm Dice, IoU, Precision và Recall.

Dataset	Model	Dice	IoU	Precision	Recall	Ghi chú
FracAtlas full test	U-Net++	0.369738	0.349318	0.368879	0.381903	613 ảnh test, gồm cả ảnh normal mask rỗng.
Ảnh có mask fracture thật	U-Net++	0.496755	0.380850	0.491880	0.565802	108 ảnh fracture trong test. Đây là metric quan trọng để đánh giá khả năng phân đoạn vùng gãy.

Theo phân loại case trên test, baseline có 36 *good case*, 48 *medium case*, 18 *hard/error case*, 6 *fracture false negative*, 173 *normal true negative* và 332 *normal false positive*. Tỷ lệ *false positive* trên ảnh *normal* là $332/505 = 0.657426$. Điều này cho thấy mô hình bắt được nhiều vùng *fracture* thật hơn no-resize, nhưng vẫn còn dự đoán nhiều trên ảnh *normal*.

7. Option no-resize / giữ kích thước ảnh gốc

Mode `no_resize_original` được dùng để kiểm tra hướng giữ nguyên kích thước ảnh gốc. Nhóm lựa chọn **không resize, không padding và không cap cạnh dài**. Nói cách khác, ảnh và mask được đưa vào model đúng như kích thước ban đầu của từng ảnh. *Cap cạnh dài* là cách thường dùng để giảm tải bộ nhớ: nếu ảnh quá lớn, ta thu nhỏ toàn bộ ảnh theo cùng một tỉ lệ sao cho cạnh dài nhất không vượt quá một ngưỡng đặt trước. Vì nhóm không cap cạnh dài, kích thước input thay đổi khá nhiều giữa các ảnh, batch size phải để bằng 1 và decoder phải giảm kênh xuống [16, 32, 64, 96] cho vừa giới hạn Colab free.

Thử nghiệm này cho thấy pipeline ảnh gốc có thể vận hành, nhưng chưa thể xem là cấu hình tối ưu. Khi giữ ảnh gốc, số pixel nền tăng lên rất nhiều trong khi vùng fracture vẫn nhỏ. Bộ nhớ GPU cũng không chỉ phụ thuộc vào ảnh đầu vào, mà còn phụ thuộc vào activation của encoder, decoder và các skip connection của U-Net++. Vì vậy no-resize dễ OOM hơn baseline resize. Nhóm áp dụng AMP, gradient accumulation, decoder checkpoint và cơ chế bỏ qua batch OOM để quá trình huấn luyện không bị dừng giữa chừng. Dù vậy, trong quá trình huấn luyện no-resize vẫn có 18 batch bị bỏ qua do OOM ở các epoch sau, nên giới hạn tài nguyên có ảnh hưởng trực tiếp đến lượng dữ liệu mô hình học được.

Thời gian chạy của no-resize cũng không ổn định: epoch đầu khoảng 236.5 giây, epoch 2 khoảng 250.1 giây, còn các epoch sau khoảng 65-68 giây nhưng vẫn có batch OOM bị bỏ qua. Sự khác biệt này đến từ kích thước ảnh không đồng nhất, cache/data loading và số batch thực tế được xử lý. Vì vậy nhóm giới hạn thử nghiệm ở 192 ảnh train, 128 ảnh validation và 160 ảnh test subset để phù hợp với Colab free. Với GPU VRAM lớn hơn, hướng này có thể được đánh giá đầy đủ hơn bằng cách tăng số mẫu, tăng decoder channels, train lâu hơn, hoặc dùng tiling/padding hợp lý.

Bảng so sánh tổng hợp giữa baseline resize và no-resize

Để người đọc dễ theo dõi, nhóm tóm tắt các khác biệt chính giữa hai hướng thực nghiệm trong Bảng 1.

Bảng 1: So sánh chi tiết giữa baseline resize và option no-resize.

Hạng mục	Baseline resize 256	No-resize original
Mục tiêu	Cấu hình chính, dùng để đánh giá segmentation trên full test.	Option thử nghiệm, kiểm tra hướng giữ kích thước ảnh gốc.
Input	Resize ảnh/mask về 256×256 , để batch và ổn định bộ nhớ.	Giữ ảnh gốc, không resize/padding/cap cạnh dài; batch size = 1.
Mô hình	U-Net++ + EfficientNet-B3; decoder [48, 96, 192, 384].	Cùng backbone, decoder giảm còn [16, 32, 64, 96] để tránh OOM.
Huấn luyện	40 epoch, batch 12, accumulation 2; train ổn định hơn.	6 epoch, batch 1, accumulation 4; có batch bị skip do OOM.
Dữ liệu test	613 ảnh: 108 fracture, 505 normal.	160 ảnh: 23 fracture, 137 normal.
Postprocess	Threshold 0.08, lọc component nhỏ và dùng <i>image-level gate</i> .	Threshold 0.03, không dùng gate để tránh xoá nhầm fracture nhỏ.
Kết quả chính	Dice full 0.369738; Dice positive 0.496755 ; Recall positive 0.565802.	Dice all 0.856250 nhưng Dice positive 0.000000 ; 23/23 fracture bị false negative.
Nhận xét	Học được tín hiệu fracture, nhưng false positive trên normal còn cao (332/505).	Pipeline chạy được, nhưng chất lượng fracture chưa đạt do subset nhỏ, decoder nhẹ và giới hạn VRAM.
Kết luận	Phù hợp làm baseline chính vì chạy đủ test và metric đáng tin hơn.	Chỉ nên xem là option thử nghiệm , cần thêm tài nguyên để đánh giá công bằng hơn.

Subset no-resize gồm 137 ảnh *normal* và 23 ảnh *fracture*. Kết quả có 137 *normal true negative* và 23 *fracture false negative*. Nói cách khác, **tất cả ảnh fracture trong subset đều bị dự đoán rỗng**. Vì vậy Dice/IoU/Precision/Recall trên nhóm ảnh có mask thật đều bằng 0. Con số *Dice all* = 0.856250 nhìn qua có vẻ cao, nhưng **không thể xem là kết quả tốt**; nó chủ yếu đến từ việc mô hình dự đoán rỗng đúng trên nhiều ảnh normal.

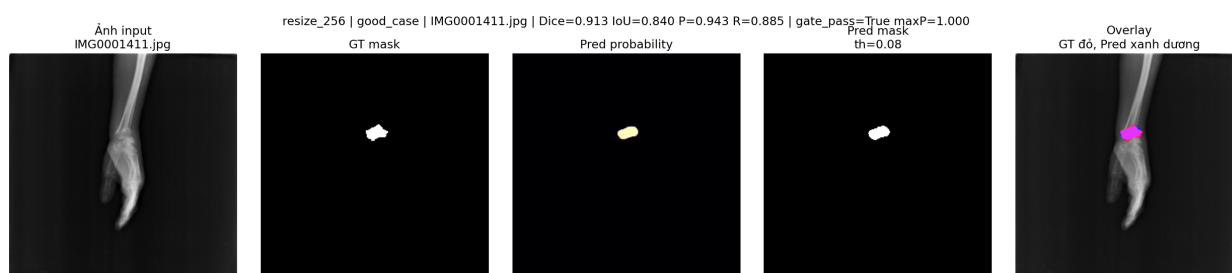
CSV kết quả và khả năng truy vết

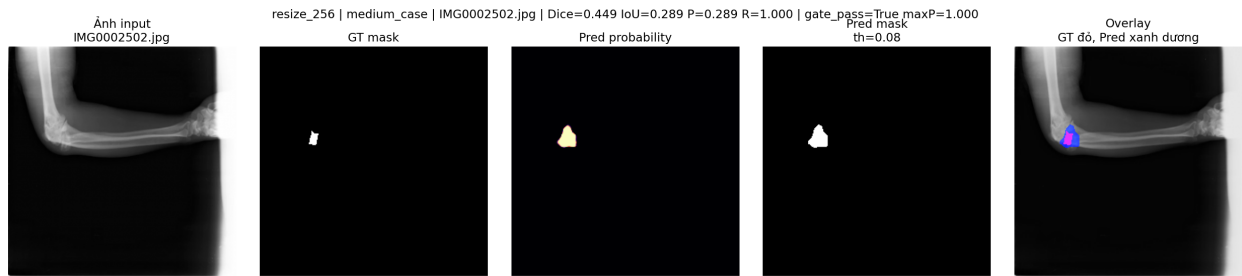
Nhóm tổng hợp kết quả vào file `results_Group07.csv` với 773 dòng: 160 dòng cho no-resize và 613 dòng cho baseline resize. Các cột chính gồm `group_id`, `student_1`, `student_2`, `task_id`, `dataset`, `model`, `image_id`, `split`, `original_width`, `original_height`, `input_width`, `input_height`, `preprocess_mode`, `gt_mask_area`, `pred_mask_area`, `dice`, `iou`, `precision`, `recall`, `case_type` và các cột khai báo AI.

Về độ bao phủ dữ liệu, baseline resize có đủ 613/613 image id trong test.csv. No-resize chỉ có 160/613 image id vì đây là subset chạy thử để giảm tải tài nguyên.

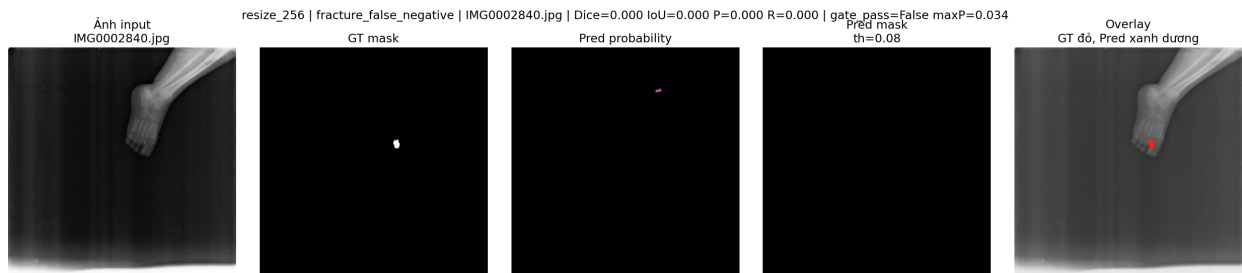
8. Hình minh họa kết quả

Các hình minh họa dưới đây được lưu trong thư mục `figures`. Mỗi hình gồm ảnh đầu vào, ground truth mask, predicted mask và overlay, giúp quan sát trực quan các trường hợp dự đoán đúng hoặc sai thay vì chỉ dựa vào metric.

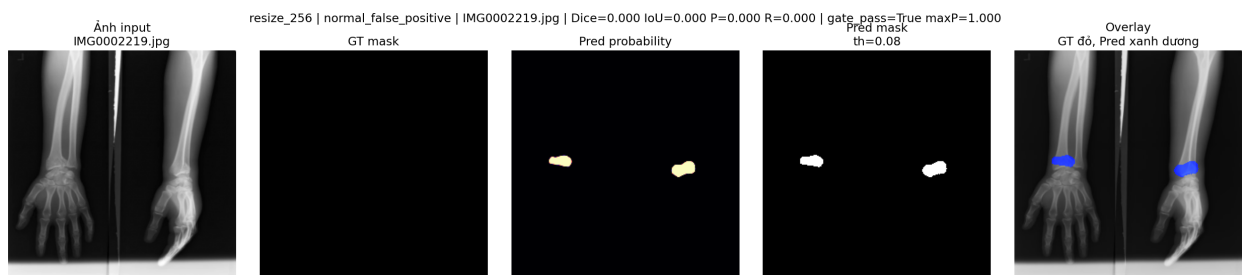
**Hình 1:** Baseline resize - good case. Mask dự đoán khớp khá tốt với vùng fracture thật.



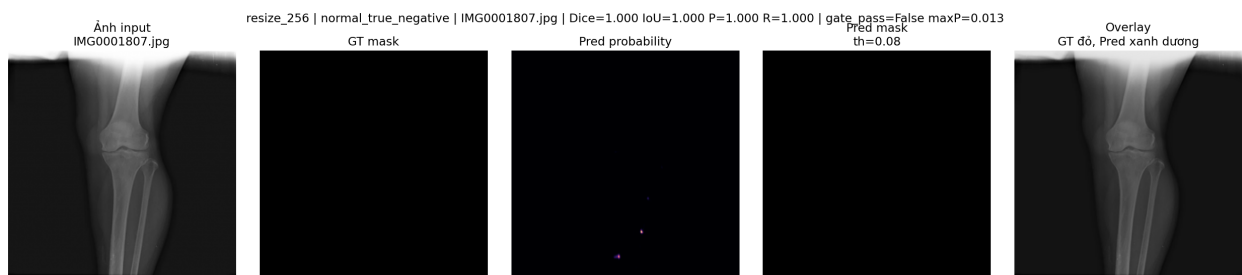
Hình 2: Baseline resize - medium case. Mô hình bắt được vùng fracture nhưng dự đoán còn rộng hơn GT.



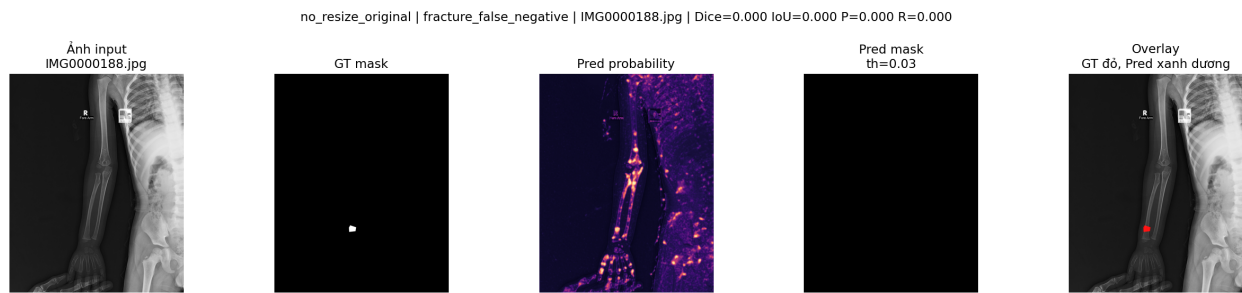
Hình 3: Baseline resize - fracture false negative. Đây là trường hợp vùng fracture bị bỏ sót.



Hình 4: Baseline resize - normal false positive. Ảnh normal nhưng mô hình vẫn sinh mask dự đoán.



Hình 5: Baseline resize - normal true negative. Ảnh normal được dự đoán rỗng đúng.



Hình 6: No-resize - fracture false negative. Mode giữ ảnh gốc vận hành được nhưng chưa phát hiện được vùng fracture trong trường hợp này.

9. Phân tích và nhận xét

Baseline resize cho thấy mô hình đã học được tín hiệu *fracture* ở mức nhất định. **Dice positive đạt 0.496755** và **Recall positive đạt 0.565802**, tức là trên nhóm ảnh có mask thật, mô hình bắt được hơn một nửa vùng *fracture* theo trung bình. Điểm yếu rõ nhất là Precision positive 0.491880 và số *normal false positive* còn cao, nghĩa là mô hình vẫn tạo nhiều mask dự đoán không cần thiết trên ảnh *normal*.

Theo nhóm, false positive có thể đến từ vài nguyên nhân. Vùng gãy thường nhỏ và biên không rõ, nên cạnh xương hoặc vùng tương phản mạnh dễ bị model nhầm là tổn thương. Ngoài ra, nhóm có oversampling ảnh fracture để model gặp nhiều positive hơn trong train; cách này giúp tăng cơ hội học fracture nhưng cũng có thể làm model nhạy quá mức với các chi tiết giống fracture. Cuối cùng, threshold 0.08 của baseline khá thấp để giữ lại fracture nhỏ, nên một số vùng nhiễu cũng bị giữ lại.

No-resize vận hành được về mặt kỹ thuật, nhưng **chất lượng chưa đạt**. Khi giữ ảnh gốc, vùng *fracture* chiếm tỷ lệ nhỏ hơn trong toàn ảnh do phần nền lớn hơn. Cấu hình no-resize lại phải giảm decoder, dùng batch size 1, train ít epoch hơn và chỉ chạy subset để tránh OOM trên Colab free. Val score đứng ở 0.30000 trong nhiều epoch; trên test subset, toàn bộ ảnh *fracture* bị *false negative*. Như vậy, ở cấu hình hiện tại, mô hình có xu hướng dự đoán rộng trên ảnh *normal* nhưng chưa học được đặc trưng của vùng *fracture*.

Dù vậy, no-resize vẫn là hướng đáng tiếp tục thử nghiệm vì nó giữ được hình dạng và tỉ lệ ảnh gốc. Nếu có tài nguyên tốt hơn, có thể tăng số mẫu, train lâu hơn, tăng decoder channels, dùng tiling hoặc padding theo batch để giảm OOM, rồi tune riêng threshold/postprocess cho no-resize. Với baseline resize, hướng cải thiện quan trọng nhất là giảm false positive, chẳng hạn bằng hard negative mining hoặc một image-level classifier/gate tốt hơn.

10. Công cụ AI hỗ trợ và cam kết

Nội dung	Khai báo của nhóm
Có sử dụng AI không?	Có.
Công cụ AI đã dùng	ChatGPT.
Phạm vi sử dụng	Hỗ trợ sửa lỗi code, diễn giải metric và rà soát lại các điểm dễ gây hiểu nhầm, comment thêm cho các dòng code. Nhóm tuyệt đối không dùng AI để bịa metric hoặc tạo hình minh họa giả.
Kết quả có truy vết từ quá trình thực nghiệm không?	Có. Metric, CSV và hình overlay đều được lưu từ quá trình chạy thực nghiệm trong <code>fracatlas_unetpp_outputs</code> .
Các thành viên có thể giải thích bài nộp không?	Có. Các thành viên nắm được pipeline dữ liệu, mô hình, loss, threshold, postprocess và hạn chế của no-resize.

11. Kết luận

Nhóm đã hoàn thành **baseline resize** cho bài toán *segmentation* ảnh X-quang với dataset `FracAtlas/segmentation` và mô hình **U-Net++** được phân công. Với encoder EfficientNet-B3 pretrained ImageNet, kết quả đạt **Dice 0.369738** trên full test và **Dice 0.496755** trên nhóm ảnh có mask *fracture* thật.

Option no-resize đã được triển khai theo hướng giữ kích thước ảnh gốc, không resize và không giới hạn cạnh dài. Giới hạn cạnh dài ở đây là thao tác thu nhỏ ảnh theo tỉ lệ khi cạnh lớn nhất vượt quá một ngưỡng cố định; nhóm có thử nghiệm bước này nhưng hiện tại thì quyết định không dùng thao tác này để kiểm tra đúng pipeline ảnh gốc. Tuy nhiên do giới hạn tài nguyên Colab free, mode này chỉ chạy subset và phải dùng cấu hình nhẹ. Kết quả no-resize hiện tại chưa đạt yêu cầu trên ảnh *fracture* vì toàn bộ 23 ảnh *fracture* trong subset bị *false negative*. **Bài học chính** là với dữ liệu có nhiều mask rỗng, cần đọc metric theo từng nhóm ảnh, đặc biệt là metric trên ảnh có tổn thương thật, thay vì chỉ nhìn Dice trung bình toàn test.

12. Checklist trước khi nộp

- `report_Group07.pdf`: xuất từ file `report_Group07.tex` bằng XeLaTeX.
- `results_Group07.csv`: file kết quả tổng hợp, không có cột index thừa, metric khớp với quá trình đánh giá.
- `notebook_link.txt`: chứa link Colab đã chạy đầy đủ và bật quyền xem.
- `PhanCong_Group07.pdf` hoặc mục phân công trong báo cáo: đã có mục phân công 50%/50%.
- `figures/`: chứa hình overlay được tạo từ quá trình chạy thật.
- File nộp Moodle cuối cùng cần đặt đúng tên `Group_07.zip` nếu nộp dạng ZIP.